UNIVERSIDAD TECNOLÓGICA DEL PERÚ

Facultad de Ingeniería
Ingeniería de Sistemas e Informática

BASE DE DATOS II

PRÁCTICA CALIFICADA N.° 2

Sistema de inventario con PostgreSQL desplegado en Railway

Docente:	Donayre Cáceres, Armando Joel
Alumno:	_____
Curso:	100000SI46 - Base de Datos II
Sección:	28969
Turno:	Noche
Sede:	Ica

Ica – Perú
13 de octubre de 2026

Índice

1. Introducción 3
1.1 Herramientas utilizadas 3
2. Pregunta 1: Modelo relacional 3
2.1 Tablas y llaves 3
2.2 Relaciones 3
2.3 Diagrama entidad-relación 4
2.4 Script SQL 4
3. Pregunta 2: Despliegue en la nube 6
3.1 Pasos que seguí 6
3.2 Datos de prueba 6
3.3 Evidencias 6
4. Pregunta 3: Inicio de sesión 8
4.1 Cómo se guardan las contraseñas 8
4.2 Cómo funciona el login 9
4.3 Pruebas 10
5. Pregunta 4: Registro de productos, entradas y salidas 11
5.1 Actualización automática del stock 11
5.2 Pantallas del sistema 12
5.3 Registro de un producto 13
5.4 Entradas y salidas 14
6. Repositorio y sistema en línea 16
6.1 Cómo ejecutarlo en local 17
7. Conclusiones 17
Anexo A: Script del esquema (01_esquema.sql) 18
Anexo B: Script de datos de prueba (02_datos_prueba.sql) 20

1. Introducción

En esta práctica desarrollé un sistema de inventario para una tienda de productos de cómputo. El sistema permite iniciar sesión, registrar productos y controlar sus entradas y salidas. La base de datos está en PostgreSQL y la desplegué en Railway; la aplicación se conecta directamente a esa base de datos en la nube.

El stock de los productos no se cambia a mano: lo calcula un **trigger** en la base de datos cada vez que se registra un movimiento. Las contraseñas de los usuarios se guardan con **hash bcrypt**, nunca en texto plano.

1.1 Herramientas utilizadas

Herramienta	Para qué la usé
PostgreSQL 18	Motor de base de datos
Railway	Servicio en la nube donde está desplegada la base de datos
Node.js y Express	Backend (API que usa la aplicación)
bcryptjs y pgcrypto	Hash de las contraseñas
JSON Web Token (JWT)	Mantener la sesión del usuario después del login
HTML, CSS y JavaScript	Pantallas del sistema
Git y GitHub	Control de versiones y entrega del código

2. Pregunta 1: Modelo relacional

Para el inventario definí cuatro tablas: **usuarios**, **categorias**, **productos** y **movimientos**. La tabla movimientos guarda tanto las entradas como las salidas; el campo `tipo` indica cuál es.

2.1 Tablas y llaves

Tabla	Llave primaria	Llaves foráneas	Descripción
usuarios	id_usuario	—	Personas que usan el sistema. Guarda el hash de la contraseña y el rol (ADMIN u OPERADOR).
categorias	id_categoria	—	Agrupa los productos: Laptops, Monitores, Redes, etc.
productos	id_producto	id_categoria	Código, nombre, precio, stock actual y stock mínimo.
movimientos	id_movimiento	id_producto, id_usuario	Cada entrada o salida con su cantidad, motivo, fecha y el stock que quedó.

2.2 Relaciones

- Una **categoría** tiene muchos **productos** (1 a N).
- Un **producto** tiene muchos **movimientos** (1 a N). Es su kardex.
- Un **usuario** registra muchos **movimientos** (1 a N).

Además de las llaves agregué restricciones `CHECK` para que el precio y el stock no sean negativos, que la cantidad de un movimiento sea mayor que cero y que el tipo solo pueda ser

ENTRADA o SALIDA. Las llaves foráneas tienen `ON DELETE RESTRICT`, así no se puede borrar un producto que ya tiene movimientos y no se pierde el historial.

2.3 Diagrama entidad-relación

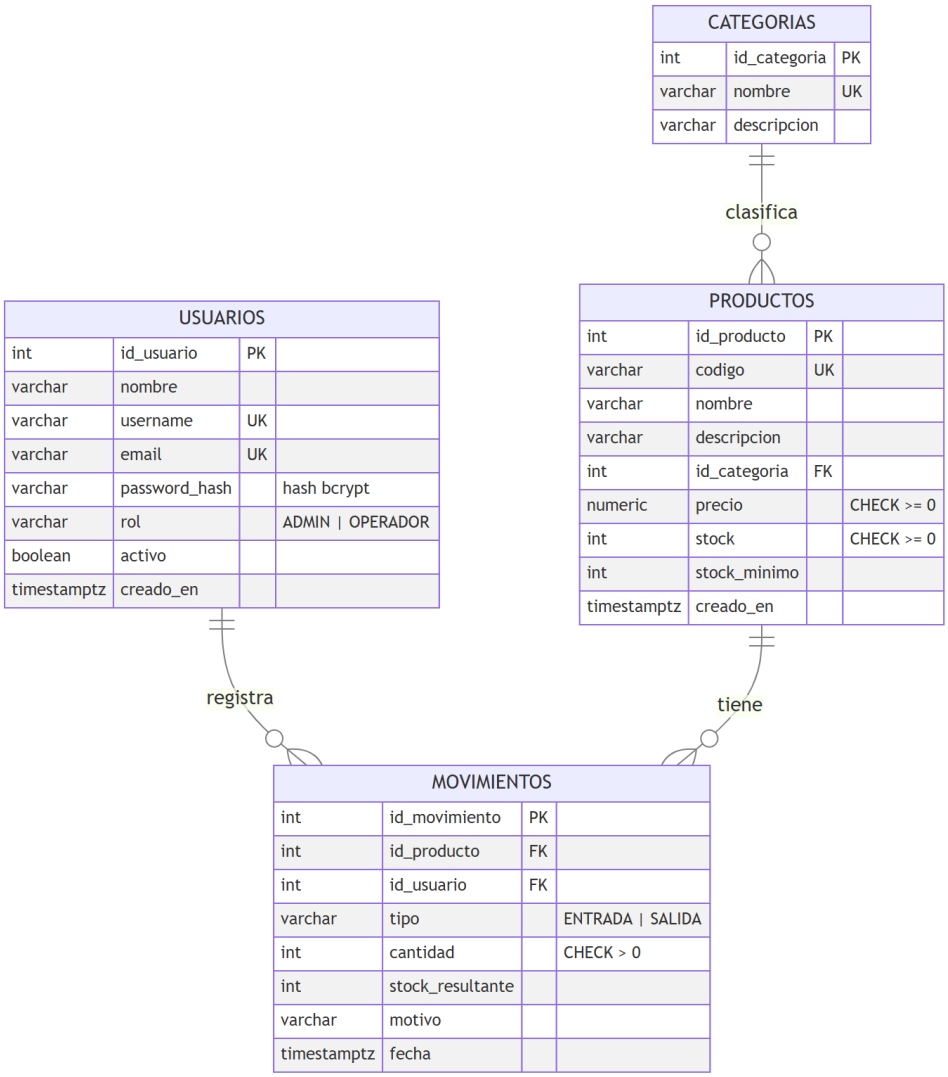


Figura 1. *Diagrama entidad-relación del sistema de inventario*

2.4 Script SQL

El script completo está en el Anexo A y en el repositorio (`database/01_esquema.sql`). Este es el fragmento de las tablas productos y movimientos:

```
CREATE TABLE productos (
    id_producto    SERIAL PRIMARY KEY,
    codigo         VARCHAR(20)   NOT NULL UNIQUE,
    nombre         VARCHAR(120)  NOT NULL,
    descripcion    VARCHAR(255),
    id_categoria   INT           NOT NULL,
    precio         NUMERIC(10,2) NOT NULL CHECK (precio >= 0),
    stock          INT           NOT NULL DEFAULT 0 CHECK (stock >= 0),
    stock_minimo   INT           NOT NULL DEFAULT 5 CHECK (stock_minimo >= 0),
    creado_en      TIMESTAMPTZ   NOT NULL DEFAULT NOW(),
    CONSTRAINT fk_productos_categoria FOREIGN KEY (id_categoria)
```

```
        REFERENCES categorias (id_categoria) ON UPDATE CASCADE ON DELETE RESTRICT
);

CREATE TABLE movimientos (
    id_movimiento    SERIAL PRIMARY KEY,
    id_producto      INT          NOT NULL,
    id_usuario       INT          NOT NULL,
    tipo             VARCHAR(10)  NOT NULL CHECK (tipo IN ('ENTRADA', 'SALIDA')),
    cantidad         INT          NOT NULL CHECK (cantidad > 0),
    stock_resultante INT,
    motivo           VARCHAR(255),
    fecha            TIMESTAMPTZ  NOT NULL DEFAULT NOW(),
    CONSTRAINT fk_movimientos_producto FOREIGN KEY (id_producto)
        REFERENCES productos (id_producto) ON UPDATE CASCADE ON DELETE RESTRICT,
    CONSTRAINT fk_movimientos_usuario FOREIGN KEY (id_usuario)
        REFERENCES usuarios (id_usuario) ON UPDATE CASCADE ON DELETE RESTRICT
);
```

3. Pregunta 2: Despliegue en la nube

Elegí **Railway** porque permite crear una base de datos PostgreSQL en pocos minutos y se puede manejar desde la terminal con su CLI. Creé un proyecto nuevo solo para esta práctica, llamado **INVENTARIO_BD2**.

3.1 Pasos que seguí

1. Inicié sesión en Railway desde la terminal con `railway login`.
2. Creé el proyecto con `railway init --name INVENTARIO_BD2`.
3. Agregué el servicio de PostgreSQL con `railway add --database postgres`.
4. Activé el **TCP Proxy** del servicio para poder conectarme desde fuera de Railway. Railway asignó el host público `thomas.proxy.rlwy.net` con el puerto `12015`.
5. Copié la cadena de conexión en el archivo `.env` del proyecto. Este archivo no se sube a GitHub porque tiene la contraseña de la base de datos.
6. Ejecuté `npm run db:init`, que corre los scripts SQL sobre la base de datos de Railway: crea las tablas, el trigger y las vistas, y carga los datos de prueba.

Resultado en la terminal:

```
Conectado a thomas.proxy.rlwy.net:12015/railway (nube)
OK 01_esquema.sql ejecutado (tablas, FKs, trigger y vistas)
OK 02_datos_prueba.sql ejecutado (usuarios, categorías, productos, movimientos)
```

(index)	tabla	registros
0	'usuarios'	2
1	'categorias'	6
2	'productos'	25
3	'movimientos'	32

3.2 Datos de prueba

Cargué 6 categorías, **25 productos**, 2 usuarios y 32 movimientos (script en el Anexo B). El stock inicial de cada producto lo registré como un movimiento de ENTRADA; así el trigger calcula el stock y queda constancia en el kardex. También agregué algunas ventas y compras de ejemplo.

3.3 Evidencias

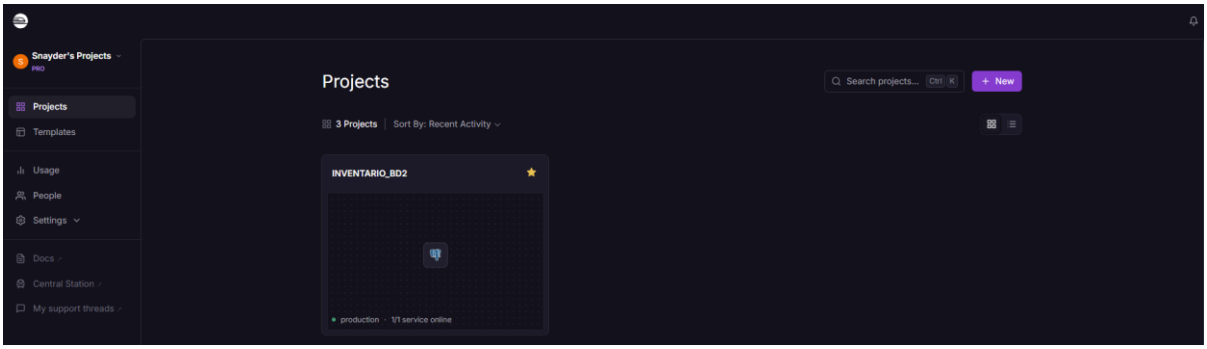


Figura 2. *Proyecto INVENTARIO_BD2 en el panel de Railway*

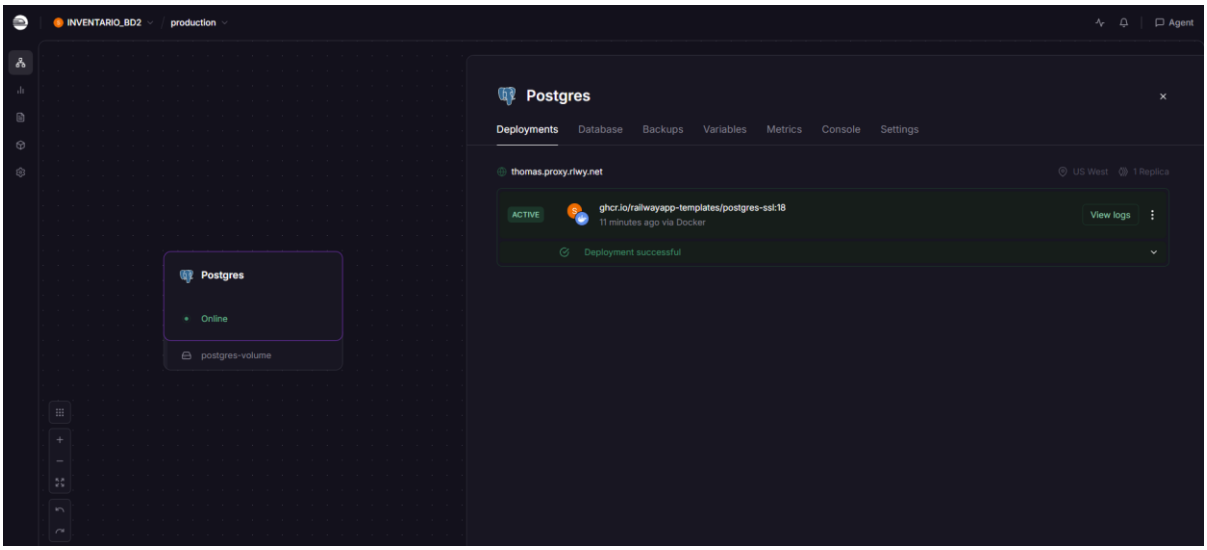


Figura 3. *Servicio Postgres desplegado y activo en Railway*

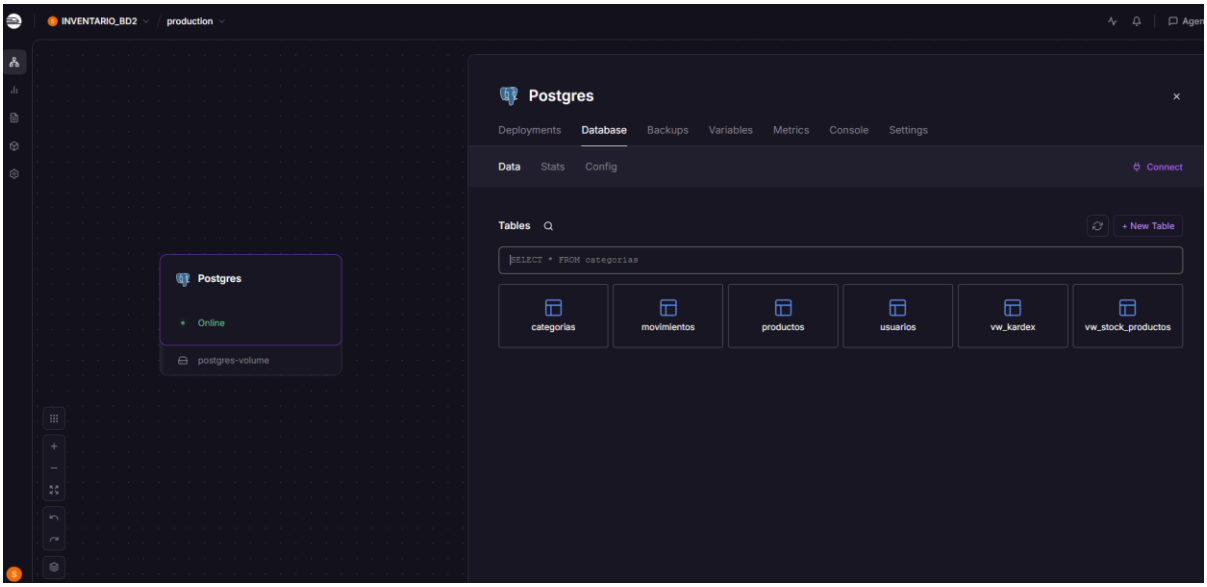


Figura 4. *Tablas y vistas creadas en la base de datos de Railway*

Consultas ejecutadas sobre la base de datos en RAILWAY (host: thomas.proxy.rlwy.net)

Servidor de base de datos

```
SELECT current_database() AS base_datos, inet_server_addr() AS ip_servidor, split_part(version(), ' on ', 1) AS version, to_char(NOW(), 'DD/MM/YYYY HH24:MI:SS') || ' (America/Lima)'
AS fecha_hora
```

base_datos	ip_servidor	version	fecha_hora
railway	10.163.60.111	PostgreSQL 18.6 (Debian 18.6-1.pgdg13+2)	13/10/2026 19:22:47 (America/Lima)

(1 filas)

Registros por tabla

```
SELECT 'usuarios' AS tabla, COUNT(*) AS registros FROM usuarios UNION ALL SELECT 'categorias', COUNT(*) FROM categorias UNION ALL SELECT 'productos', COUNT(*) FROM productos UNION
ALL SELECT 'movimientos', COUNT(*) FROM movimientos
```

tabla	registros
usuarios	2
categorias	6
productos	25
movimientos	32

(4 filas)

Contraseñas almacenadas con hash bcrypt

```
SELECT id_usuario, username, rol, password_hash FROM usuarios ORDER BY id_usuario
```

id_usuario	username	rol	password_hash
1	admin	ADMIN	$2a$10$cZ5lkzeBe0Q4YW8QBdmxBOz7tGCg5DtOYNVaVJo3h2vEyp5d35Su6
2	operador	OPERADOR	$2a$10$q.wMVzjLVzP5diDIkZeBZOxdKs48hfR26zmuP/PzVAEHUvLk/g44.

(2 filas)

Productos cargados (stock calculado por el trigger)

```
SELECT codigo, nombre, categoria, precio, stock, stock_minimo, stock_bajo FROM vw_stock_productos ORDER BY codigo
```

codigo	nombre	categoria	precio	stock	stock_minimo	stock_bajo
ALM-001	SSD Kingston NV2 1 TB	Almacenamiento	259.00	25	6	false
ALM-002	SSD Samsung 870 EVO 500 GB	Almacenamiento	239.00	20	6	false
ALM-003	Disco Externo Seagate 2 TB	Almacenamiento	329.00	10	4	false
ALM-004	USB SanDisk 64 GB	Almacenamiento	35.00	60	15	false
COM-001	Memoria RAM Kingston 8 GB	Componentes	119.00	30	8	false
COM-002	Memoria RAM Corsair 16 GB	Componentes	229.00	20	6	false
COM-003	Procesador AMD Ryzen 5 5600	Componentes	599.00	2	3	true
COM-004	Fuente EVGA 600 W 80+	Componentes	259.00	8	3	false
COM-005	Pasta Térmica Arctic MX-4	Componentes	29.90	40	10	false
LAP-001	Laptop Lenovo IdeaPad 5	Laptops	2899.00	7	3	false
LAP-002	Laptop HP Pavilion 15	Laptops	2499.00	12	3	false
LAP-003	Laptop ASUS VivoBook 14	Laptops	1799.00	8	3	false
LAP-004	Laptop Dell Inspiron 3520	Laptops	3599.00	5	2	false
MON-001	Monitor LG 24" IPS	Monitores	549.00	10	4	false
MON-002	Monitor Samsung 27" Curvo	Monitores	799.00	9	3	false
MON-003	Monitor AOC 24G2 144 Hz	Monitores	899.00	7	3	false
PER-001	Mouse Logitech M170	Periféricos	39.90	70	10	false
PER-002	Teclado Logitech K120	Periféricos	49.90	45	10	false
PER-003	Audífonos HyperX Cloud II	Periféricos	349.00	15	5	false
PER-004	Webcam Logitech C920	Periféricos	299.00	12	5	false
PER-005	Mouse Pad Redragon XL	Periféricos	45.00	30	8	false
RED-001	Router TP-Link Archer C6	Redes	189.00	15	4	false
RED-002	Switch TP-Link 8 puertos	Redes	119.00	12	4	false
RED-003	Cable UTP Cat6 3 m	Redes	12.00	110	20	false
RED-004	Adaptador WiFi USB TP-Link	Redes	79.00	18	5	false

(25 filas)

Figura 5. *Consultas ejecutadas sobre la base de datos de Railway: servidor, registros por tabla, contraseñas con hash y los 25 productos*

4. Pregunta 3: Inicio de sesión

4.1 Cómo se guardan las contraseñas

Las contraseñas no se guardan en texto plano. En la columna `password_hash` se guarda el resultado de **bcrypt**, que incluye una sal aleatoria; por eso dos usuarios con la misma clave tienen hashes distintos y no se puede obtener la clave original a partir del hash.

- Los usuarios de prueba los inserté desde SQL con `crypt('clave', gen_salt('bf', 10))` de la extensión pgcrypto.
- Los usuarios que se crean desde el sistema se guardan con `bcrypt.hash(password, 10)` en el backend.

En la Figura 5 se ve la columna password_hash con valores como `$2a$10$...`, que es el formato de bcrypt.

4.2 Cómo funciona el login

1. El usuario escribe su usuario y contraseña en la pantalla de inicio.
2. El backend busca al usuario en la base de datos de Railway con una consulta parametrizada.
3. Con `bcrypt.compare()` se compara la contraseña escrita con el hash guardado.
4. Si es correcta, el servidor devuelve un token JWT que dura 8 horas. La aplicación lo envía en cada petición; sin el token, la API responde con error 401.
5. Si es incorrecta, se muestra "Usuario o contraseña incorrectos", sin decir cuál de los dos datos falló.

Código del login (`src/routes/auth.js`):

```
router.post('/login', async (req, res) => {
  const { username, password } = req.body || {};
  if (!username || !password) {
    return res.status(400).json({ error: 'Usuario y contraseña son obligatorios' });
  }

  const { rows } = await pool.query(
    `SELECT id_usuario, nombre, username, password_hash, rol, activo
       FROM usuarios WHERE username = $1`,
    [username.trim()]
  );
  const usuario = rows[0];

  const valido = usuario && usuario.activo && (await bcrypt.compare(password,
usuario.password_hash));
  if (!valido) {
    return res.status(401).json({ error: 'Usuario o contraseña incorrectos' });
  }

  const payload = { id: usuario.id_usuario, username: usuario.username, nombre:
usuario.nombre, rol: usuario.rol };
  const token = jwt.sign(payload, process.env.JWT_SECRET, { expiresIn: '8h' });
  res.json({ token, usuario: payload });
});
```

4.3 Pruebas

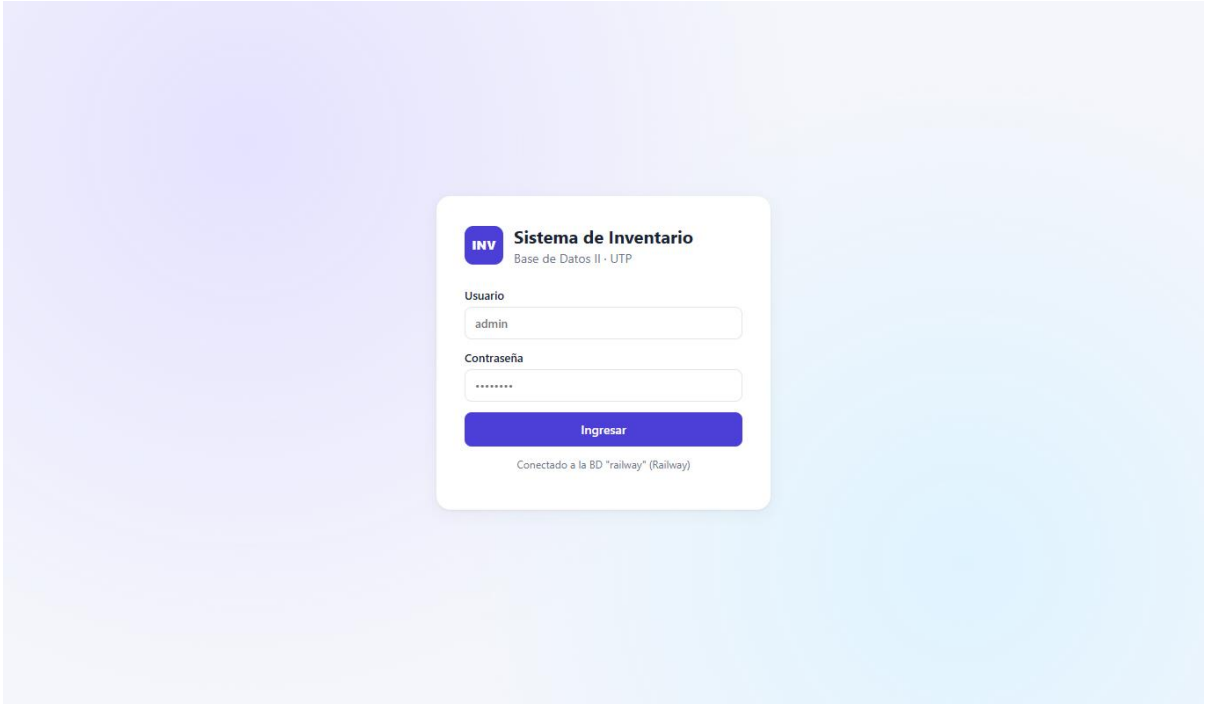


Figura 6. *Pantalla de inicio de sesión conectada a la base de datos de Railway*

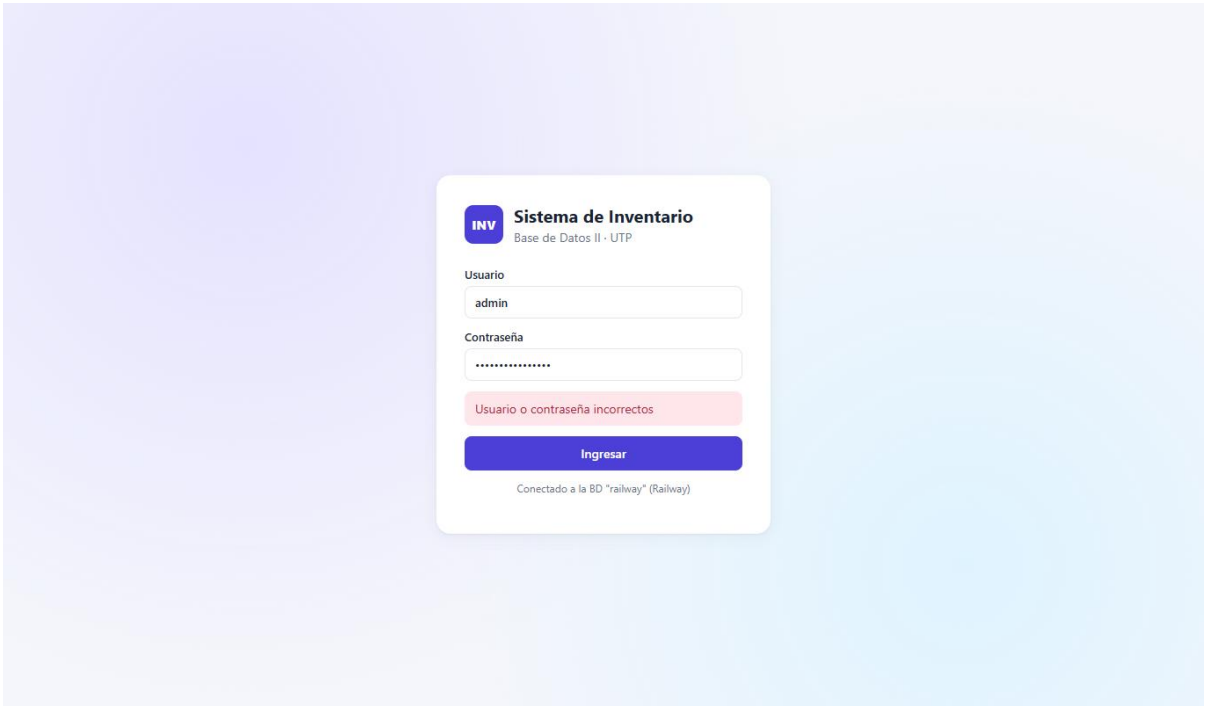


Figura 7. *Intento de ingreso con una contraseña incorrecta*

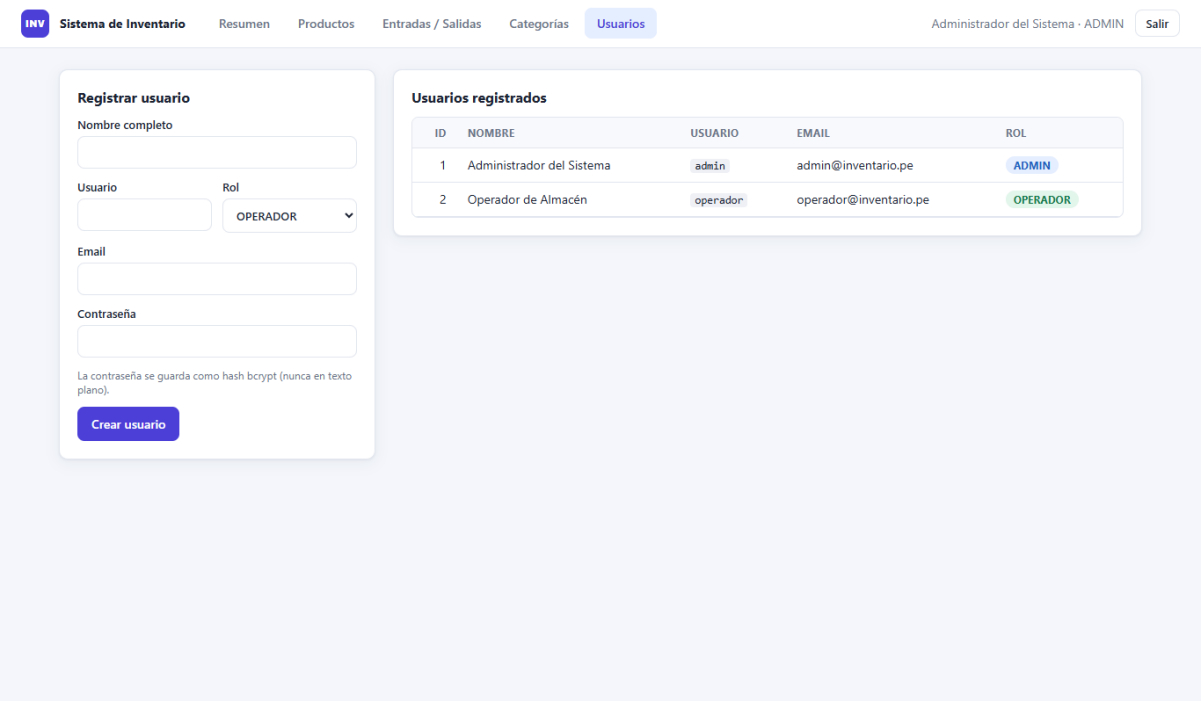


Figura 8. *Módulo de usuarios (solo ADMIN): al crear un usuario su contraseña se guarda con hash*

5. Pregunta 4: Registro de productos, entradas y salidas

5.1 Actualización automática del stock

Para que el stock se actualice solo creé un trigger en PostgreSQL. Se ejecuta antes de insertar cada movimiento: si es una entrada suma la cantidad, si es una salida la resta, y si no alcanza el stock lanza un error y la salida no se guarda.

```
CREATE OR REPLACE FUNCTION fn_actualizar_stock()
RETURNS TRIGGER AS $$
DECLARE
    v_stock INT;
BEGIN
    SELECT stock INTO v_stock
      FROM productos
     WHERE id_producto = NEW.id_producto
       FOR UPDATE;

    IF NEW.tipo = 'ENTRADA' THEN
        v_stock := v_stock + NEW.cantidad;
    ELSE
        IF v_stock < NEW.cantidad THEN
            RAISE EXCEPTION 'Stock insuficiente: disponible %, solicitado %', v_stock,
NEW.cantidad;
        END IF;
        v_stock := v_stock - NEW.cantidad;
    END IF;

    UPDATE productos SET stock = v_stock WHERE id_producto = NEW.id_producto;
    NEW.stock_resultante := v_stock;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_movimientos_stock
    BEFORE INSERT ON movimientos
    FOR EACH ROW
```

```
    EXECUTE FUNCTION fn_actualizar_stock();
```

La aplicación solo inserta el movimiento; el backend nunca actualiza el stock directamente. El `FOR UPDATE` bloquea la fila del producto mientras se calcula, para que dos salidas al mismo tiempo no dejen el stock mal. Cuando se registra un producto con stock inicial, el producto y su primera entrada se guardan en una misma transacción (BEGIN / COMMIT).

5.2 Pantallas del sistema

Al ingresar se muestra un resumen con los totales del inventario y los productos que están en su stock mínimo o por debajo.

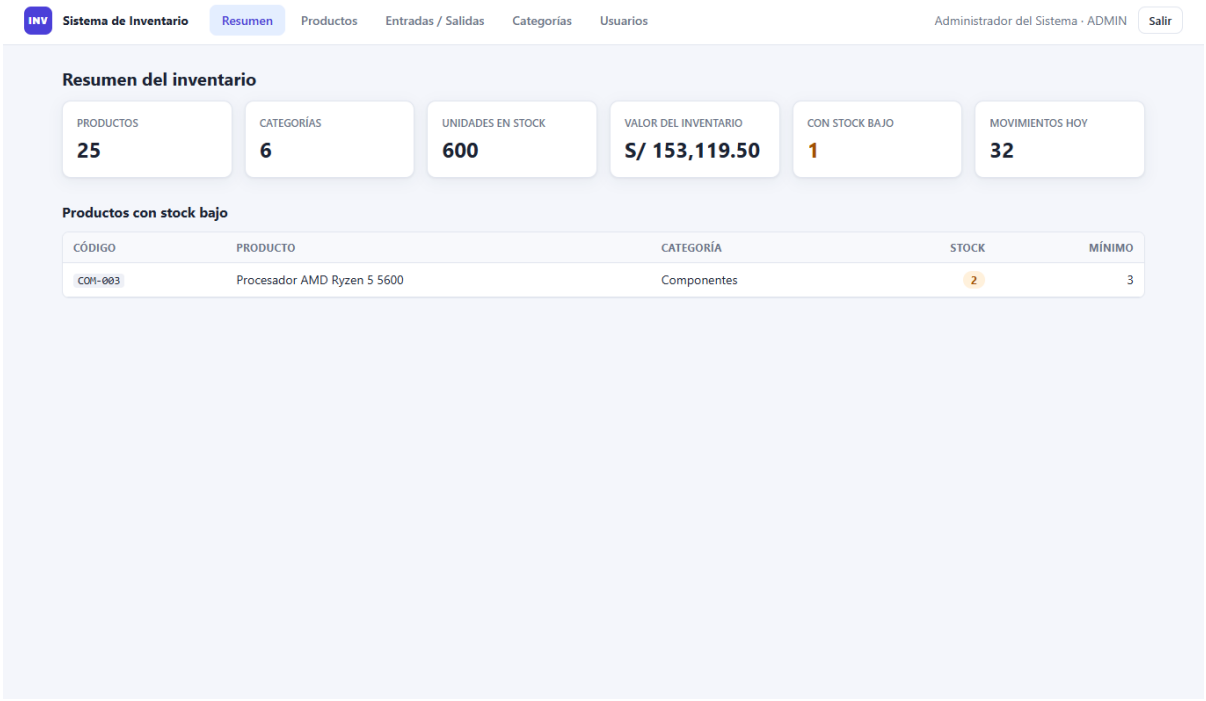


Figura 9. *Resumen del inventario*

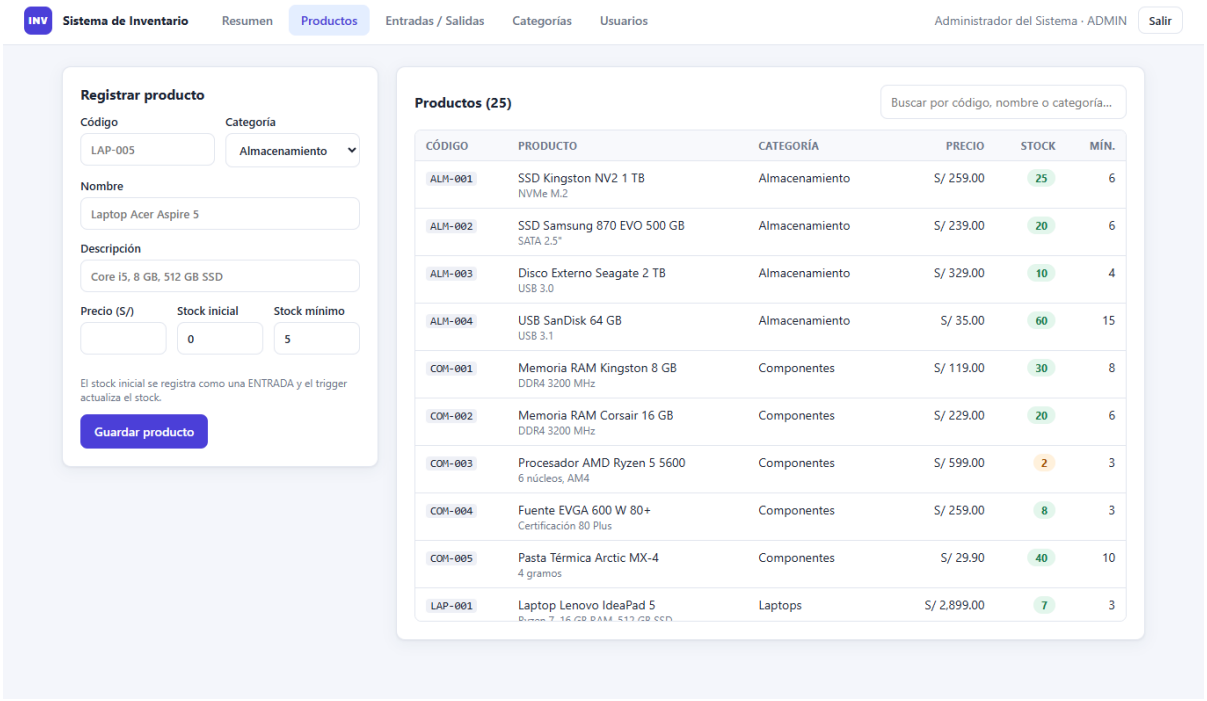


Figura 10. *Listado de productos con su categoría, precio y stock*

5.3 Registro de un producto

Registré una laptop Acer Aspire 5 con 6 unidades de stock inicial. Esas 6 unidades entran como un movimiento de ENTRADA y el trigger deja el stock en 6.

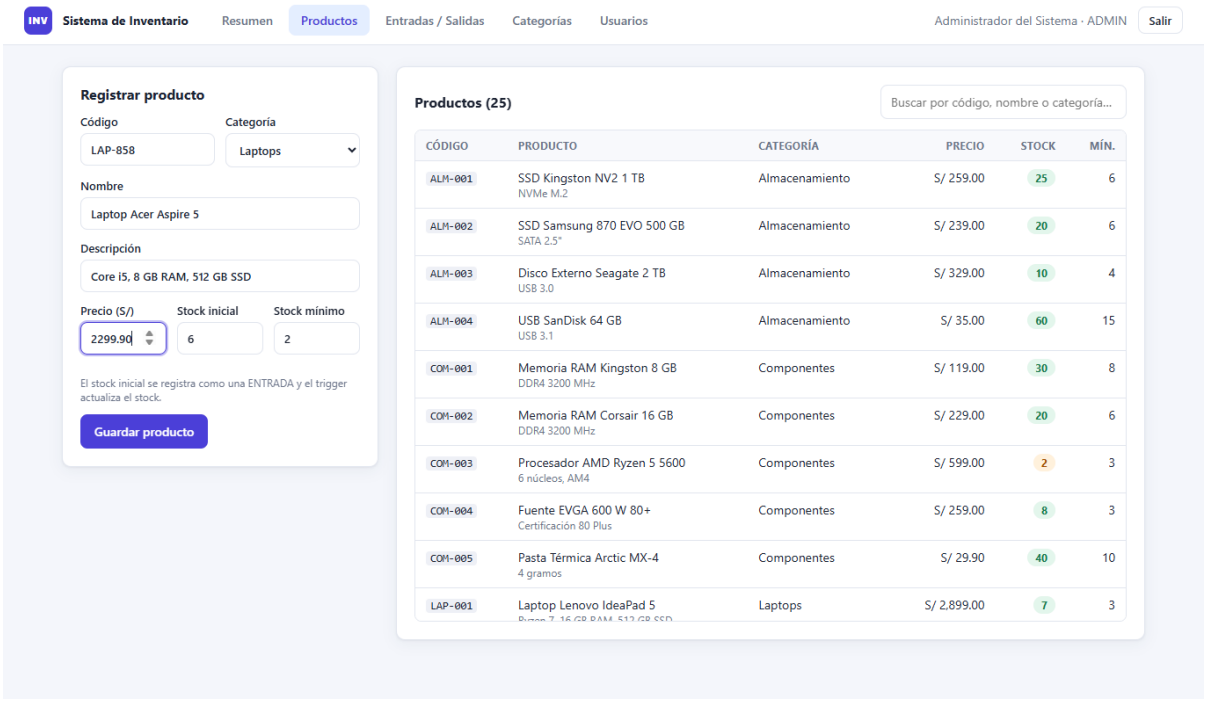


Figura 11. *Formulario de registro de producto*

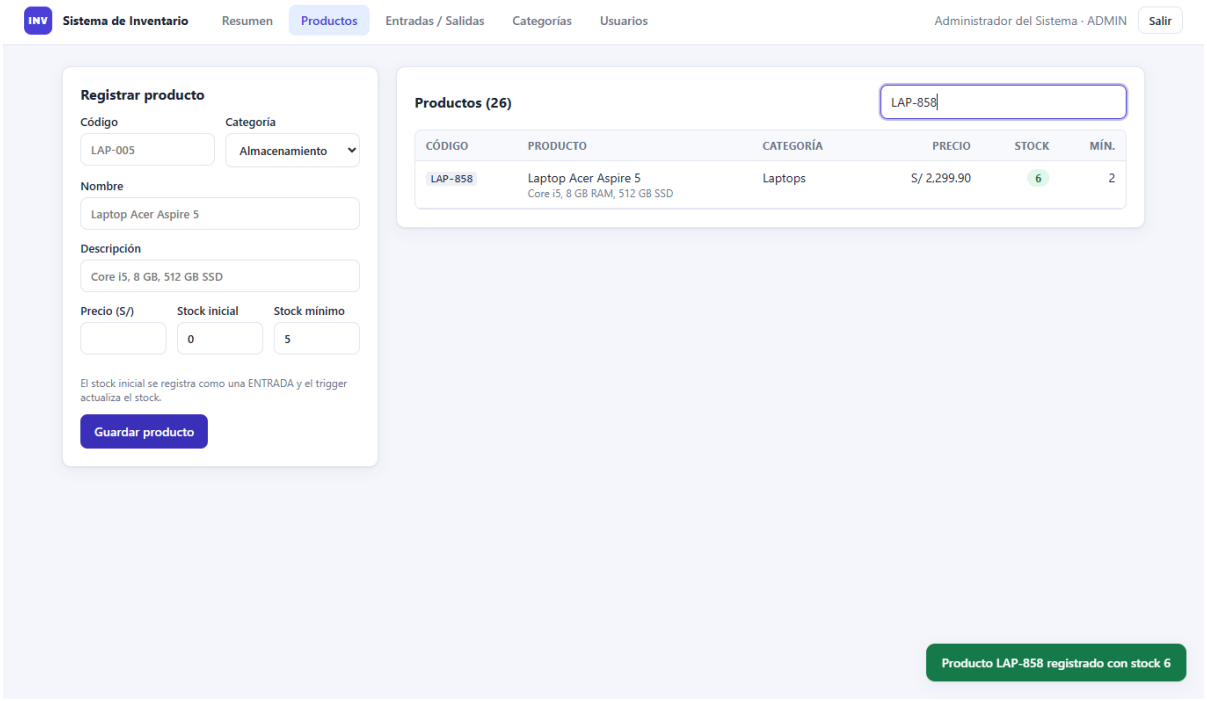


Figura 12. *Producto registrado con stock 6*

5.4 Entradas y salidas

Luego registré una entrada de 10 unidades por una compra al proveedor: el stock pasó de 6 a **16**.

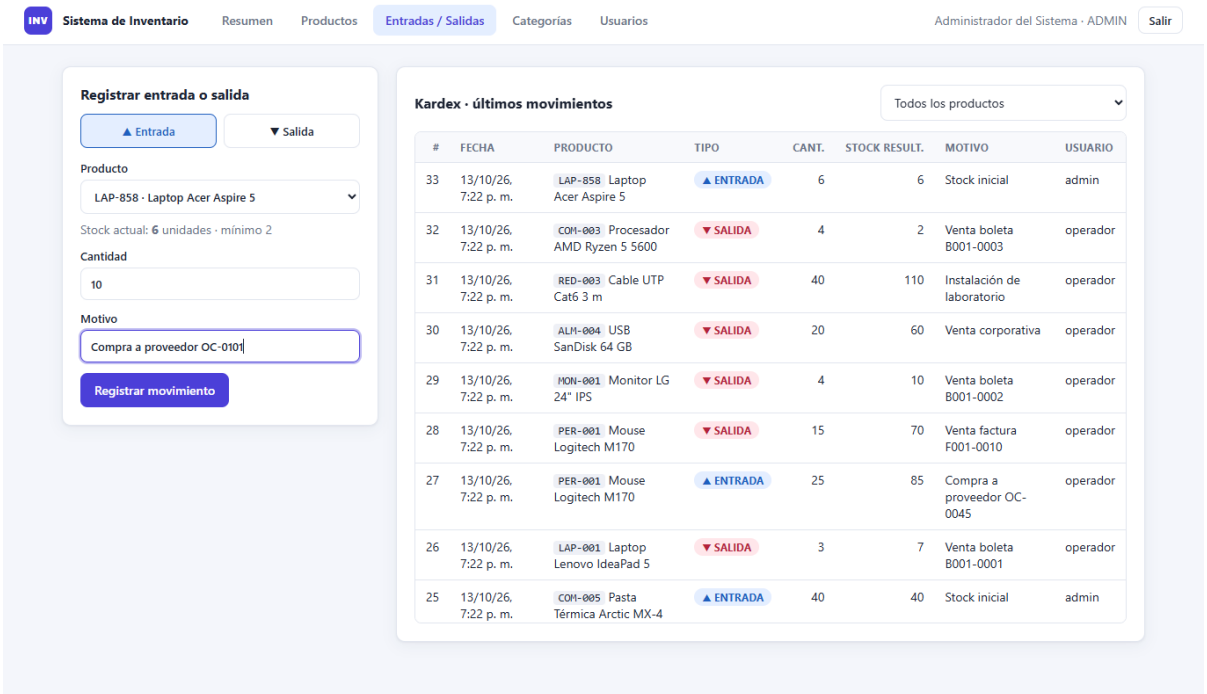


Figura 13. *Registro de una entrada de 10 unidades*

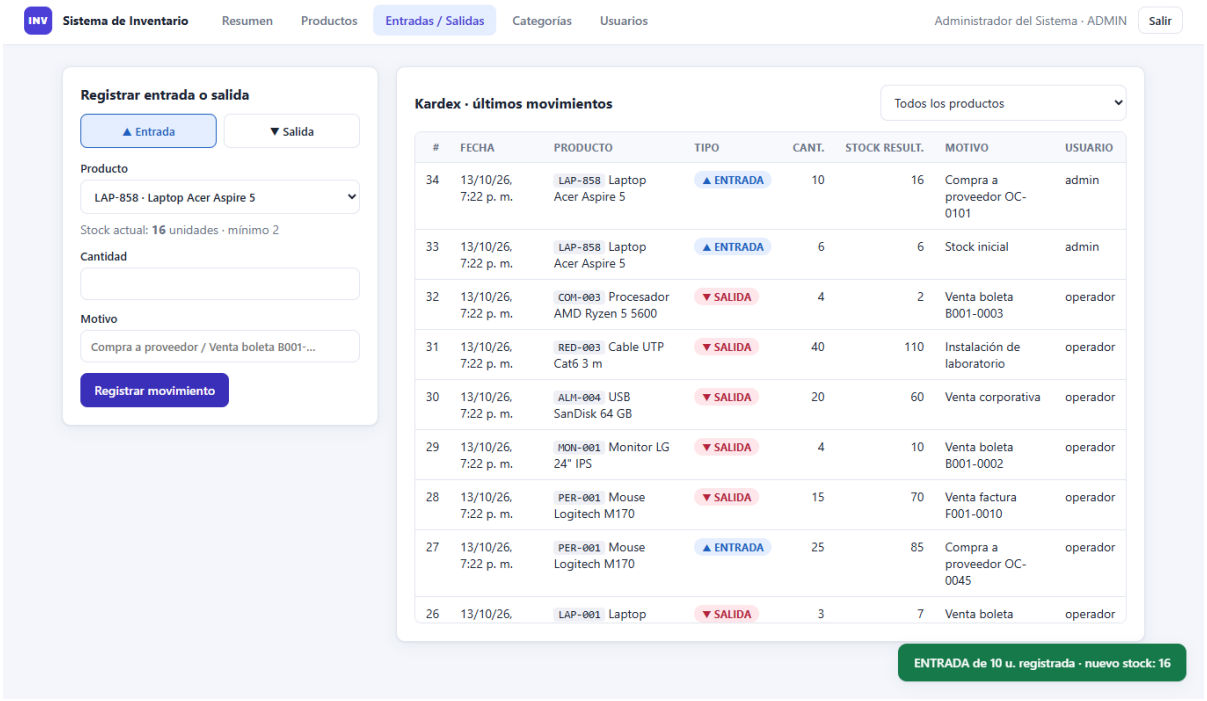


Figura 14. *Entrada registrada: el stock sube a 16 y aparece en el kardex*

Después registré una salida de 4 unidades por una venta: el stock bajó a **12**.

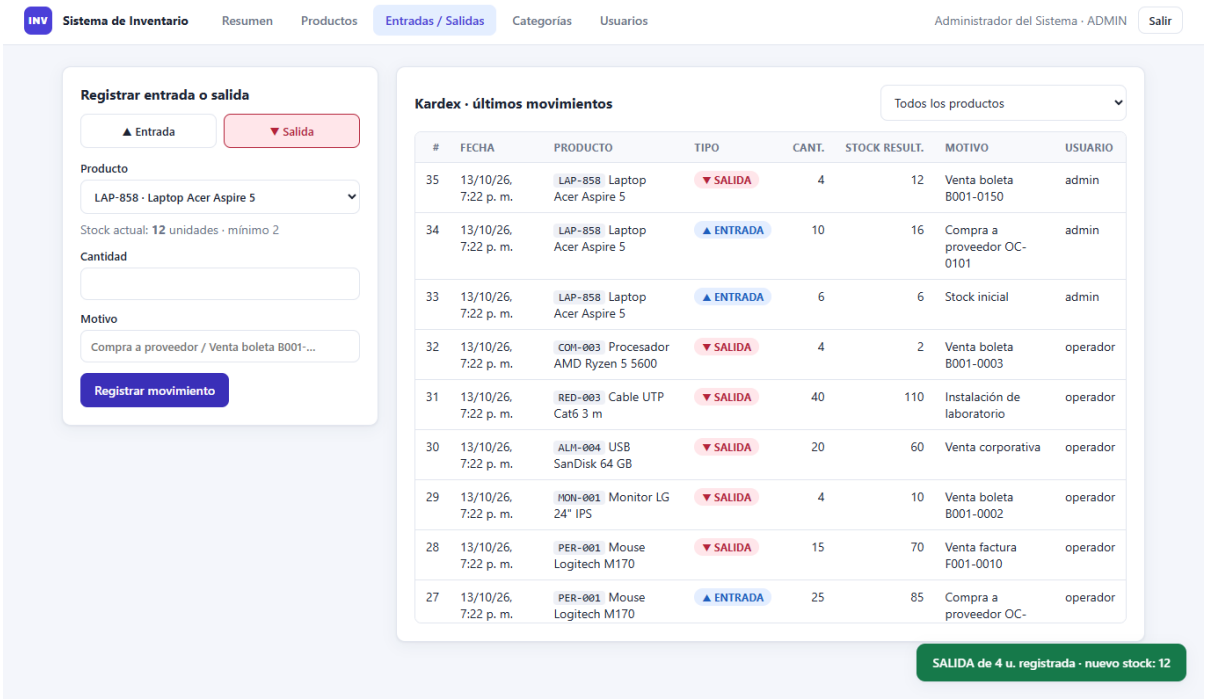


Figura 15. *Salida registrada: el stock baja a 12*

Por último intenté sacar 500 unidades. El trigger rechazó la operación porque solo había 12 y el movimiento no se guardó.

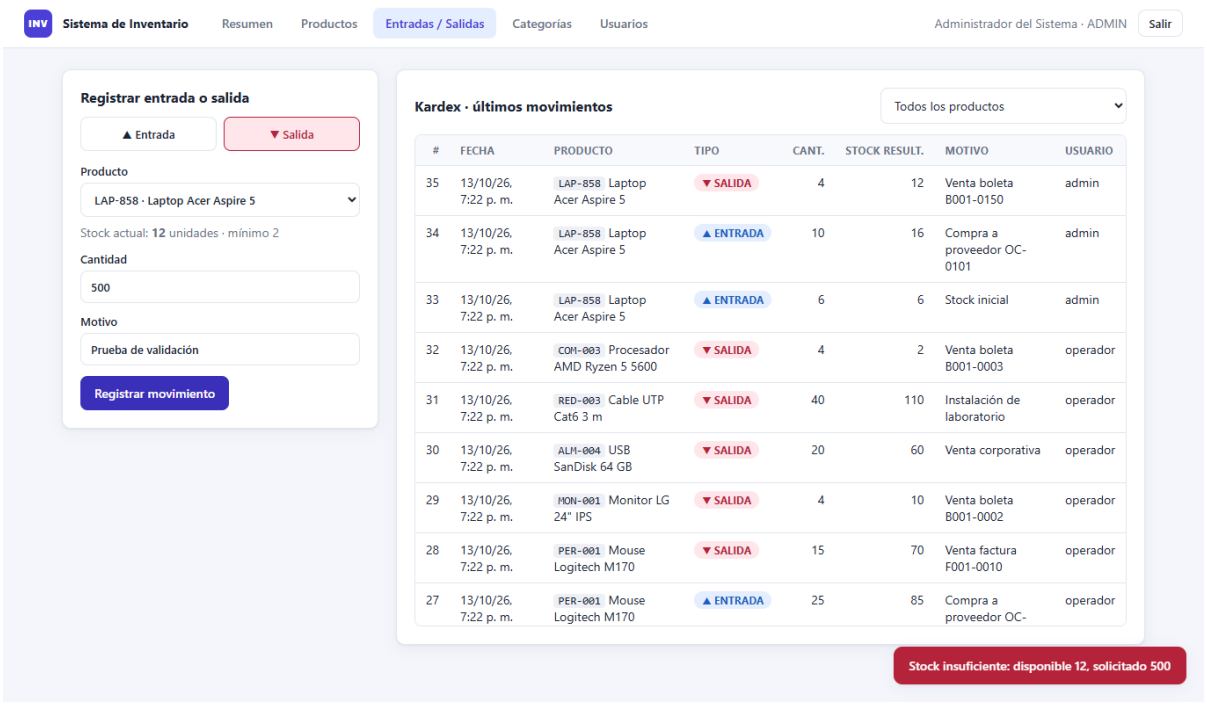


Figura 16. *Salida rechazada por stock insuficiente*

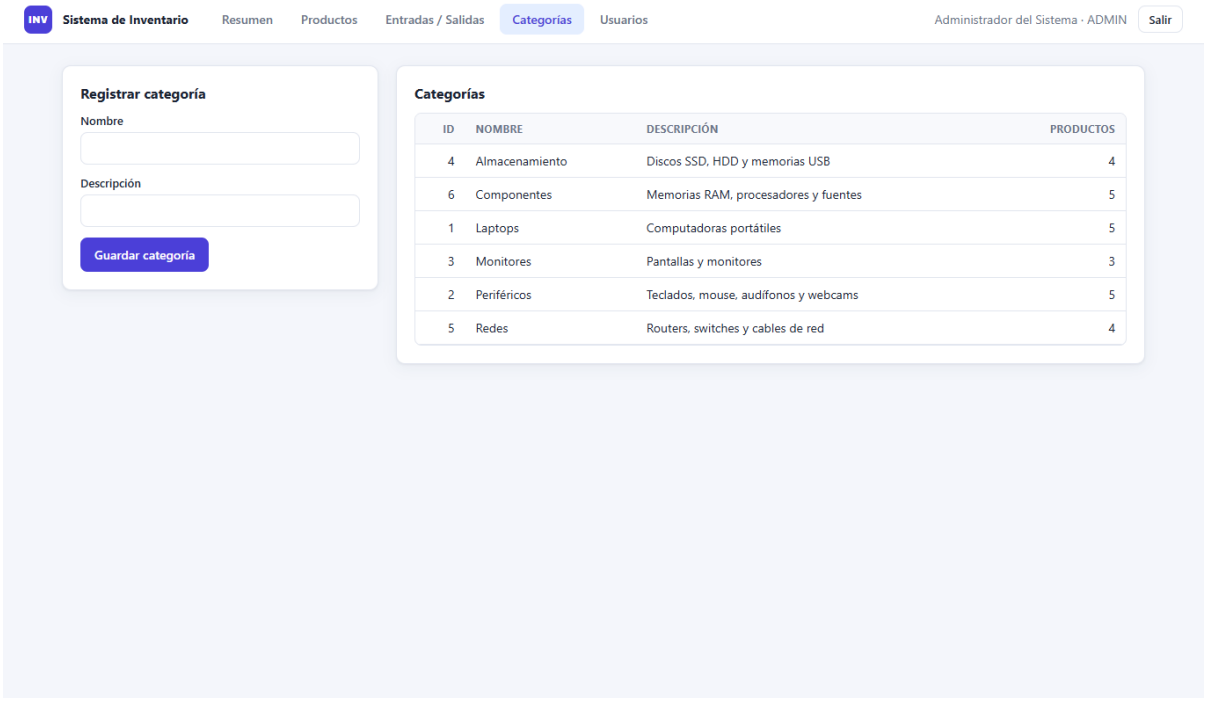


Figura 17. *Módulo de categorías*

6. Repositorio y sistema en línea

El código fuente, los scripts SQL y las capturas están en GitHub:

https://github.com/CahuanaSnayder/inventario-bd2-pc2

Además subí el sistema de inventario al mismo proyecto de Railway como un servicio web (**inventario-web**). Se conecta a la base de datos por la red privada de Railway y se puede abrir desde cualquier navegador:

https://inventario-web-production-89a8.up.railway.app

Carpeta / archivo	Contenido
database/	Scripts SQL: esquema, datos de prueba y consultas de verificación
src/	Backend en Node.js y Express (login, productos, movimientos)
public/	Pantallas del sistema (HTML, CSS y JavaScript)
scripts/	Script para crear la base de datos y cargar los datos
docs/	Diagrama ER y capturas
README.md	Documentación y pasos para ejecutar el proyecto

6.1 Cómo ejecutarlo en local

1. Instalar dependencias con `npm install`.
2. Copiar `.env.example` como `.env` y colocar la cadena de conexión de Railway (`DATABASE_URL`) y una clave para `JWT_SECRET`.
3. Crear las tablas y cargar los datos con `npm run db:init`.
4. Iniciar el servidor con `npm start` y abrir http://localhost:3000.

Usuarios de prueba (sirven en local y en la versión en línea):

Usuario	Contraseña	Rol
admin	Admin2026!	ADMIN
operador	Operador2026!	OPERADOR

7. Conclusiones

- Poner la regla del stock en un trigger hace que no dependa de la aplicación: aunque alguien inserte un movimiento directamente con SQL, el stock se actualiza igual y nunca queda negativo.
- Guardar solo el hash de la contraseña protege a los usuarios si alguien llega a ver la tabla, porque no se puede recuperar la clave original.
- Las llaves foráneas con RESTRICT evitan borrar productos o usuarios que ya tienen movimientos, así no se pierde el historial del inventario.
- Railway hizo sencillo el despliegue. Lo único adicional fue activar el TCP Proxy para conectarme a la base de datos desde mi computadora.

Anexo A: Script del esquema (01_esquema.sql)

```
-- Practica Calificada 2 - Base de Datos II
-- Sistema de inventario en PostgreSQL

CREATE EXTENSION IF NOT EXISTS pgcrypto;  -- para crypt() y gen_salt()

-- Si ya existen, se borran para poder volver a ejecutar el script
DROP VIEW IF EXISTS vw_kardex;
DROP VIEW IF EXISTS vw_stock_productos;
DROP TABLE IF EXISTS movimientos CASCADE;
DROP TABLE IF EXISTS productos CASCADE;
DROP TABLE IF EXISTS categorias CASCADE;
DROP TABLE IF EXISTS usuarios CASCADE;
DROP FUNCTION IF EXISTS fn_actualizar_stock();



CREATE TABLE usuarios (
    id_usuario     SERIAL PRIMARY KEY,
    nombre         VARCHAR(100) NOT NULL,
    username       VARCHAR(50)  NOT NULL UNIQUE,
    email          VARCHAR(120) NOT NULL UNIQUE,
    password_hash  VARCHAR(255) NOT NULL,      -- se guarda el hash, no la clave
    rol            VARCHAR(20)  NOT NULL DEFAULT 'OPERADOR' CHECK (rol IN ('ADMIN', 'OPERADOR')),
    activo         BOOLEAN      NOT NULL DEFAULT TRUE,
    creado_en      TIMESTAMPTZ  NOT NULL DEFAULT NOW()
);

CREATE TABLE categorias (
    id_categoria   SERIAL PRIMARY KEY,
    nombre         VARCHAR(80) NOT NULL UNIQUE,
    descripcion    VARCHAR(255)
);

CREATE TABLE productos (
    id_producto    SERIAL PRIMARY KEY,
    codigo         VARCHAR(20)   NOT NULL UNIQUE,
    nombre         VARCHAR(120)  NOT NULL,
    descripcion    VARCHAR(255),
    id_categoria   INT           NOT NULL,
    precio         NUMERIC(10,2) NOT NULL CHECK (precio >= 0),
    stock          INT           NOT NULL DEFAULT 0 CHECK (stock >= 0),
    stock_minimo   INT           NOT NULL DEFAULT 5 CHECK (stock_minimo >= 0),
    creado_en      TIMESTAMPTZ   NOT NULL DEFAULT NOW(),
    CONSTRAINT fk_productos_categoria FOREIGN KEY (id_categoria)
        REFERENCES categorias (id_categoria) ON UPDATE CASCADE ON DELETE RESTRICT
);

-- Entradas y salidas de cada producto (kardex)
CREATE TABLE movimientos (
    id_movimiento    SERIAL PRIMARY KEY,
    id_producto      INT          NOT NULL,
    id_usuario       INT          NOT NULL,
    tipo             VARCHAR(10)  NOT NULL CHECK (tipo IN ('ENTRADA', 'SALIDA')),
    cantidad         INT          NOT NULL CHECK (cantidad > 0),
    stock_resultante INT,
    motivo           VARCHAR(255),
    fecha            TIMESTAMPTZ  NOT NULL DEFAULT NOW(),
    CONSTRAINT fk_movimientos_producto FOREIGN KEY (id_producto)
        REFERENCES productos (id_producto) ON UPDATE CASCADE ON DELETE RESTRICT,
    CONSTRAINT fk_movimientos_usuario FOREIGN KEY (id_usuario)
        REFERENCES usuarios (id_usuario) ON UPDATE CASCADE ON DELETE RESTRICT
);

CREATE INDEX idx_productos_categoria  ON productos (id_categoria);
CREATE INDEX idx_movimientos_producto ON movimientos (id_producto);
CREATE INDEX idx_movimientos_usuario  ON movimientos (id_usuario);
CREATE INDEX idx_movimientos_fecha    ON movimientos (fecha DESC);



-- Trigger: cada vez que se inserta un movimiento se actualiza el stock.
-- Si es una salida y no alcanza el stock, se cancela la operacion.
CREATE OR REPLACE FUNCTION fn_actualizar_stock()
RETURNS TRIGGER AS $$
```

```
DECLARE
    v_stock INT;
BEGIN
    SELECT stock INTO v_stock
      FROM productos
     WHERE id_producto = NEW.id_producto
       FOR UPDATE;

    IF NOT FOUND THEN
        RAISE EXCEPTION 'El producto % no existe', NEW.id_producto;
    END IF;

    IF NEW.tipo = 'ENTRADA' THEN
        v_stock := v_stock + NEW.cantidad;
    ELSE
        IF v_stock < NEW.cantidad THEN
            RAISE EXCEPTION 'Stock insuficiente: disponible %, solicitado %', v_stock, NEW.cantidad
                USING ERRCODE = 'check_violation';
        END IF;
        v_stock := v_stock - NEW.cantidad;
    END IF;

    UPDATE productos SET stock = v_stock WHERE id_producto = NEW.id_producto;
    NEW.stock_resultante := v_stock;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_movimientos_stock
    BEFORE INSERT ON movimientos
    FOR EACH ROW
    EXECUTE FUNCTION fn_actualizar_stock();



-- Vistas para los reportes
CREATE VIEW vw_stock_productos AS
SELECT p.id_producto, p.codigo, p.nombre, c.nombre AS categoria,
       p.precio, p.stock, p.stock_minimo,
       (p.stock <= p.stock_minimo) AS stock_bajo,
       (p.precio * p.stock) AS valor_inventario
  FROM productos p
  JOIN categorias c ON c.id_categoria = p.id_categoria;

CREATE VIEW vw_kardex AS
SELECT m.id_movimiento, m.fecha, p.codigo, p.nombre AS producto,
       m.tipo, m.cantidad, m.stock_resultante, m.motivo, u.username AS usuario
  FROM movimientos m
  JOIN productos p ON p.id_producto = m.id_producto
  JOIN usuarios u ON u.id_usuario = m.id_usuario;
```

Anexo B: Script de datos de prueba (02_datos_prueba.sql)

```
-- Datos de prueba: usuarios, categorias, 25 productos y algunos movimientos


-- Las claves se guardan como hash bcrypt con crypt() de pgcrypto
INSERT INTO usuarios (nombre, username, email, password_hash, rol) VALUES
('Administrador del Sistema', 'admin',    'admin@inventario.pe',    crypt('Admin2026!',    gen_salt('bf', 10)), 'ADMIN'),
('Operador de Almacén',       'operador', 'operador@inventario.pe', crypt('Operador2026!', gen_salt('bf', 10)),
'OPERADOR');


INSERT INTO categorias (nombre, descripcion) VALUES
('Laptops',        'Computadoras portátiles'),
('Periféricos',    'Teclados, mouse, audífonos y webcams'),
('Monitores',      'Pantallas y monitores'),
('Almacenamiento', 'Discos SSD, HDD y memorias USB'),
('Redes',          'Routers, switches y cables de red'),
('Componentes',    'Memorias RAM, procesadores y fuentes');


-- El stock empieza en 0, se carga despues con movimientos de entrada
INSERT INTO productos (codigo, nombre, descripcion, id_categoria, precio, stock_minimo) VALUES
('LAP-001', 'Laptop Lenovo IdeaPad 5',      'Ryzen 7, 16 GB RAM, 512 GB SSD',  1, 2899.00, 3),
('LAP-002', 'Laptop HP Pavilion 15',        'Core i5, 8 GB RAM, 512 GB SSD',   1, 2499.00, 3),
('LAP-003', 'Laptop ASUS VivoBook 14',      'Core i3, 8 GB RAM, 256 GB SSD',   1, 1799.00, 3),
('LAP-004', 'Laptop Dell Inspiron 3520',    'Core i7, 16 GB RAM, 1 TB SSD',    1, 3599.00, 2),
('PER-001', 'Mouse Logitech M170',          'Inalámbrico 2.4 GHz',             2,   39.90, 10),
('PER-002', 'Teclado Logitech K120',        'USB, español',                    2,   49.90, 10),
('PER-003', 'Audífonos HyperX Cloud II',    'Gaming 7.1',                      2,  349.00, 5),
('PER-004', 'Webcam Logitech C920',         'Full HD 1080p',                   2,  299.00, 5),
('PER-005', 'Mouse Pad Redragon XL',        '80 x 30 cm',                      2,   45.00, 8),
('MON-001', 'Monitor LG 24" IPS',           '1920x1080, 75 Hz',                3,  549.00, 4),
('MON-002', 'Monitor Samsung 27" Curvo',    '1920x1080, 75 Hz',                3,  799.00, 3),
('MON-003', 'Monitor AOC 24G2 144 Hz',      'Gaming, IPS',                     3,  899.00, 3),
('ALM-001', 'SSD Kingston NV2 1 TB',        'NVMe M.2',                        4,  259.00, 6),
('ALM-002', 'SSD Samsung 870 EVO 500 GB',   'SATA 2.5"',                       4,  239.00, 6),
('ALM-003', 'Disco Externo Seagate 2 TB',   'USB 3.0',                         4,  329.00, 4),
('ALM-004', 'USB SanDisk 64 GB',            'USB 3.1',                         4,   35.00, 15),
('RED-001', 'Router TP-Link Archer C6',     'AC1200 doble banda',              5,  189.00, 4),
('RED-002', 'Switch TP-Link 8 puertos',     'Gigabit no administrable',        5,  119.00, 4),
('RED-003', 'Cable UTP Cat6 3 m',           'Patch cord',                      5,   12.00, 20),
('RED-004', 'Adaptador WiFi USB TP-Link',   'AC600',                           5,   79.00, 5),
('COM-001', 'Memoria RAM Kingston 8 GB',    'DDR4 3200 MHz',                   6,  119.00, 8),
('COM-002', 'Memoria RAM Corsair 16 GB',    'DDR4 3200 MHz',                   6,  229.00, 6),
('COM-003', 'Procesador AMD Ryzen 5 5600',  '6 núcleos, AM4',                  6,  599.00, 3),
('COM-004', 'Fuente EVGA 600 W 80+',        'Certificación 80 Plus',           6,  259.00, 3),
('COM-005', 'Pasta Térmica Arctic MX-4',    '4 gramos',                        6,   29.90, 10);


-- Stock inicial (el trigger actualiza productos.stock)
INSERT INTO movimientos (id_producto, id_usuario, tipo, cantidad, motivo)
SELECT p.id_producto, 1, 'ENTRADA', s.cantidad, 'Stock inicial'
  FROM productos p
  JOIN (VALUES
        ('LAP-001', 10), ('LAP-002', 12), ('LAP-003', 8),  ('LAP-004', 5),
        ('PER-001', 60), ('PER-002', 45), ('PER-003', 15), ('PER-004', 12),
        ('PER-005', 30), ('MON-001', 14), ('MON-002', 9),  ('MON-003', 7),
        ('ALM-001', 25), ('ALM-002', 20), ('ALM-003', 10), ('ALM-004', 80),
        ('RED-001', 15), ('RED-002', 12), ('RED-003', 150),('RED-004', 18),
        ('COM-001', 30), ('COM-002', 20), ('COM-003', 6),  ('COM-004', 8),
        ('COM-005', 40)
       ) AS s(codigo, cantidad) ON s.codigo = p.codigo
 ORDER BY p.id_producto;


-- Algunas ventas y compras de ejemplo
INSERT INTO movimientos (id_producto, id_usuario, tipo, cantidad, motivo)
SELECT p.id_producto, 2, s.tipo, s.cantidad, s.motivo
  FROM productos p
  JOIN (VALUES
        ('LAP-001', 'SALIDA',  3, 'Venta boleta B001-0001'),
        ('PER-001', 'SALIDA', 15, 'Venta factura F001-0010'),
        ('MON-001', 'SALIDA',  4, 'Venta boleta B001-0002'),
        ('ALM-004', 'SALIDA', 20, 'Venta corporativa'),
        ('COM-003', 'SALIDA',  4, 'Venta boleta B001-0003'),
        ('PER-001', 'ENTRADA', 25, 'Compra a proveedor OC-0045'),
        ('RED-003', 'SALIDA', 40, 'Instalación de laboratorio')
       ) AS s(codigo, tipo, cantidad, motivo) ON s.codigo = p.codigo;
```